



Laporan Praktikum Algoritma & Pemrograman
Semester Genap 2025/2026

SAYA MENYATAKAN BAHWA LAPORAN PRAKTIKUM INI SAYA BUAT DENGAN USAHA SENDIRI TANPA MENGGUNAKAN BANTUAN ORANG LAIN. SEMUA MATERI YANG SAYA AMBIL DARI SUMBER LAIN SUDAH SAYA CANTUMKAN SUMBERNYA DAN TELAH SAYA TULIS ULANG DENGAN BAHASA SAYA SENDIRI.

SAYA SANGGUP MENERIMA SANKSI JIKA MELAKUKAN KEGIATAN PLAGIASI, TERMASUK SANKSI TIDAK LULUS MATA KULIAH INI.

NIM	71251235
Nama Lengkap	NATALIA GRECIA PUTRI
Minggu ke / Materi	12 / Tipe Data Tuples

PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS KRISTEN DUTA WACANA
YOGYAKARTA
2026

BAGIAN 1: MATERI MINGGU INI (40%)

Pada bagian ini, tuliskan kembali semua materi yang telah anda pelajari minggu ini. Sesuaikan penjelasan anda dengan urutan materi yang telah diberikan di saat praktikum. Penjelasan anda harus dilengkapi dengan contoh, gambar/ilustrasi, contoh program (source code) dan outputnya. Idealnya sekitar 5-6 halaman.

Tuple Immutable

Tuple memiliki kemiripan dengan list dalam hal kemampuannya menyimpan berbagai tipe data dan mengakses elemen melalui indeks bilangan bulat. Meski demikian, keduanya memiliki perbedaan mendasar: tuple bersifat immutable, yang berarti nilai-nilai di dalamnya tidak dapat dimodifikasi setelah tuple tersebut dibuat. Di samping itu, tuple mendukung operasi perbandingan antar nilai dan memiliki sifat hashable, sehingga dapat digunakan sebagai kunci (key) dalam struktur data dictionary di Python.

Suatu objek dikategorikan sebagai hashable apabila ia memiliki nilai hash yang konsisten selama masa hidupnya—dihasilkan oleh metode `__hash__()`—dan dapat dibandingkan dengan objek lain melalui metode seperti `__eq__()` atau `__cmp__()`. Sifat hashable memungkinkan objek untuk difungsikan sebagai kunci dalam dictionary atau sebagai anggota dalam set, mengingat kedua struktur data tersebut bergantung pada mekanisme hash secara internal. Pada umumnya, objek bawaan Python seperti bilangan dan string bersifat hashable, sementara struktur data yang dapat diubah seperti list dan dictionary tidak demikian. Untuk kelas yang dibuat secara manual, objek-objeknya secara default tidak hashable kecuali metode `__hash__()` didefinisikan secara eksplisit.

```
1  #Membuat tuple tanpa tanda kurung, Python otomatis mengenali sebagai tuple
2  t = 'a', 'b', 'c', 'd', 'e'
3  print("Tuple t tanpa kurung:", t)
4
5  #Membuat tuple tanpa tanda kurung (Lebih jelas dibaca)
6  t = ('a', 'b', 'c', 'd', 'e')
7  print("Tuple t tanpa kurung:", t)
8
9  #tuple dengan satu elemen harus diakhiri dengan koma
10 t1 = ('a',)
11 print("Tuple t1 (satu elemen): ", t1)
12 print("Tipe t1:", type(t1)) # Akan mencetak: <class 'tuple'>
13
14 # Jika tidak memakai koma, maka dianggap sebagai string biasa, bukan tuple
15 t2 = ('a')
16 print("t2 tanpa koma:", t2)
17 print("Tipe t1:", type(t2)) # Akan mencetak: <class 'str'>
18
19 #Membuat tuple kosong menggunakan fungsi bawaan tuple()
20 t_empty = tuple()
21 print("Tuple kosong:", t_empty)
```

```

1 #Membuat tuple dari sebuah string, setiap karakter akan menjadi elemen tuple
2 t_string = tuple('dutaawacana')
3 print("Tuple dari string 'dutaawacana': ", t_string)
4
5 #Indexing pada tuple, mengambil elemen pertama (indeks dimulai dari 0)
6 t = ('a', 'b', 'c', 'd', 'e')
7 print("Elemen pertama:", t[0])
8
9 #Slicing tuple, mengambil elemen dari indeks 1 sampai sebelum indeks 3
10 print("Elemen ke-2 dan ke-3:", t[1:3])
11
12 #Tuple bersifat immutable, artinya elemen tidak bisa diubah langsung
13 try:
14     t[0] = 'A' #ini akan menghasilkan TypeError karena tuple tidak bisa diubah
15 except TypeError as e:
16     print("Error saat menggunakan elemen tuple:", e)
17
18 #Untuk 'mengubah' isi tuple, kita buat tuple baru dengan elemen yang diinginkan
19 #Di sini, elemen pertama pertama diganti dengan 'A' dan digabung dengan elemen sisanya
20 t = ('A',) + t[1:]
21 print("Tuple setelah 'mengubah' elemen pertama:", t)

```

Membandingkan Tuple

Operator perbandingan dalam Python dapat diterapkan pada tuple maupun struktur sekuensial lainnya seperti list, dictionary, dan set. Proses perbandingan dimulai dari elemen pertama di masing-masing struktur. Apabila elemen pertama bernilai sama, perbandingan akan dilanjutkan ke elemen berikutnya secara berurutan, hingga ditemukan elemen yang berbeda sebagai penentu hasil akhir.

Fungsi sort() di Python menerapkan logika serupa: pengurutan dimulai dari elemen pertama, dan jika nilainya sama, dilanjutkan ke elemen berikutnya. Untuk kasus yang lebih rumit, Python menggunakan teknik yang disebut DSU (Decorate, Sort, Undecorate) dengan tiga tahapan:

- Decorate – Setiap elemen dalam urutan dikombinasikan dengan satu atau lebih kunci pengurutan, biasanya dalam bentuk pasangan tuple.
- Sort – Daftar tuple yang terbentuk kemudian diurutkan menggunakan fungsi sort() Python berdasarkan kunci yang telah ditambahkan.
- Undecorate – Setelah pengurutan selesai, kunci tambahan dihilangkan sehingga daftar kembali berisi elemen aslinya.

```

kalimat = 'but soft what light in yonder window breaks'
dalkata = kalimat.split()
t = list()
for kata in dalkata:
    t.append((len(kata), kata))

t.sort(reverse=True)

urutan = list()
for length, kata in t:
    urutan.append(kata)

print(urutan)

```

Sebagai contoh penerapannya, teknik ini dapat digunakan untuk mengurutkan kata-kata dalam sebuah kalimat berdasarkan panjangnya. Pada iterasi pertama, Python membentuk daftar tuple berisi pasangan (panjang kata, kata), lalu mengurutkannya berdasarkan elemen pertama (panjang kata). Jika dua kata memiliki panjang yang sama, perbandingan dilanjutkan ke elemen kedua. Penggunaan parameter `reverse=True` akan menghasilkan urutan dari kata terpanjang ke yang terpendek. Pada iterasi kedua, daftar disusun berdasarkan abjad secara menurun dengan tetap mempertimbangkan panjang kata, sehingga kata seperti "what" akan muncul lebih dahulu dibanding "soft".

Penugasan *Tuple*

Salah satu fitur menarik Python adalah kemampuan menempatkan tuple di sisi kiri dari operasi penugasan (assignment). Fitur ini memungkinkan penetapan beberapa variabel secara bersamaan dalam satu baris instruksi. Python akan memecah elemen-elemen dari list atau tuple di sisi kanan berdasarkan posisi indeksinya, lalu menetapkannya satu per satu ke variabel yang sesuai di sisi kiri.

Fitur ini sangat praktis digunakan, misalnya untuk menukar nilai dua variabel hanya dengan satu baris kode. Python memperlakukan kedua sisi penugasan sebagai tuple, sehingga setiap nilai di sisi kanan langsung dipasangkan ke variabel yang bersesuaian di sisi kiri. Perlu diperhatikan bahwa jumlah elemen di kedua sisi harus identik; jika tidak, Python akan memunculkan error. Penerapan lain yang umum adalah memisahkan username dari nama domain dalam sebuah alamat email menggunakan metode `split()`.

```
1 # Membuat list dengan dua elemen
2 m = ['have', 'fun']
3
4 # Multiple assignment (penugasan ganda) menggunakan tuple tanpa tanda kurung
5 x, y = m
6 print("x:", x) # Output: "have"
7 print("y:", y) # Output: "fun"
8
9 # Penugasan di atas sama dengan menuliskan:
10 x = m[0]
11 y = m[1]
12 print("Ulangi penugasan manual:")
13 print("x:", x)
14 print("y:", y)
15
16 # Contoh lain: menggunakan tanda kurung untuk menunjukkan tuple secara eksplisit
17 m = ['have', 'fun']
18 (x, y) = m # Ini sama dengan sebelumnya
19 print("Dengan tanda kurung:")
20 print("x:", x)
21 print("y:", y)
```

```

1 # Menukar nilai dua variabel dalam satu baris dengan tuple assignment
2 a = 10
3 b = 20
4 print("Sebelum ditukar: a =", a, ", b =", b)
5 a, b = b, a
6 print("Setelah ditukar: a =", a, ", b =", b)
7
8 # Kesalahan jika jumlah variabel tidak sesuai dengan jumlah elemen
9 try:
10     a, b = 1, 2, 3 # Ini akan error
11 except ValueError as e:
12     print("Error:", e)
13
14 #Contoh praktis: memisahkan email menjadi username dan domain
15 email = 'didanendya@ti.ukdw.ac.id'
16 username, domain = email.split('@') #split() mengembalikan List ['didanendya', 'ti.ukdw.ac.id']
17
18 print("Username:", username) #Output: "didanendya
19 print("Domain:", domain) #Output: "ti.ukdw.ac.id

```

Dictionaries and Tuple

Dictionary di Python menyediakan metode `items()` yang menghasilkan daftar berisi pasangan tuple, di mana masing-masing tuple terdiri dari satu key dan satu value. Perlu diketahui bahwa keluaran dari `items()` tidak memiliki urutan tertentu karena dictionary tidak menjamin urutan penyimpanan berdasarkan waktu input. Namun demikian, karena hasilnya berupa list berisi tuple, dan tuple mendukung operasi perbandingan, daftar ini dapat diurutkan. Dengan mengonversi dictionary menjadi list of tuples terlebih dahulu, kita bisa menampilkan isi dictionary yang tersusun rapi berdasarkan kuncinya.

```

1 # Membuat dictionary dengan beberapa pasangan key-value
2 d = {'a': 10, 'b': 1, 'c': 22}
3
4 # Menggunakan metode items() untuk mengambil daftar tuple dari dictionary
5 t = list(d.items()) # Konversi hasil items() menjadi List
6 print("List tuple dari dictionary (sebelum diurutkan):")
7 print(t) # Output: [('b', 1), ('a', 10), ('c', 22)] tidak berurutan
8
9 # Mengurutkan List dari tuple berdasarkan key (default sort pada elemen pertama dari tuple)
10 t.sort()
11 print("List tuple setelah diurutkan berdasarkan key:")
12 print(t) # Output: [('a', 10), ('b', 1), ('c', 22)]
13
14 # Penjelasan tambahan:
15 # Fungsi sort() akan mengurutkan berdasarkan elemen pertama dari tuple (yaitu key)
16 # Karena key adalah string, maka urutannya secara alfabet (ascending)
17
18 # Jika ingin mengurutkan berdasarkan value, bisa gunakan parameter key:
19 t_by_value = sorted(d.items(), key=lambda item: item[1])
20 print("\nList tuple diurutkan berdasarkan value (nilai):")
21 print(t_by_value) #Output: [('b', 1), ('a', 10), ('c', 22)]

```

Multipenugasan dengan dictionaries

Python memungkinkan penggunaan teknik tuple assignment saat melakukan iterasi pada dictionary. Ketika metode `items()` dipanggil, hasilnya berupa daftar tuple yang masing-masing merepresentasikan pasangan key-value. Dalam proses perulangan, setiap pasangan tuple tersebut dapat langsung dipecah ke dalam dua variabel terpisah, misalnya key dan val. Dengan pendekatan ini, key dan value dapat diakses sekaligus secara efisien dalam satu blok loop. Apabila diperlukan pengurutan berdasarkan value, kita dapat membuat list baru berisi tuple dengan

format (value, key), lalu mengurutkannya. Karena perbandingan tuple dimulai dari elemen pertama, pengurutan secara otomatis akan didasarkan pada value.

```
1 # Membuat dictionary dengan beberapa pasangan key-value
2 d = {'a': 10, 'b': 1, 'c': 22}
3
4 # Menggunakan metode items() untuk mengambil daftar tuple dari dictionary
5 t = list(d.items()) # Konversi hasil items() menjadi list
6 print("List tuple dari dictionary (sebelum diurutkan):")
7 print(t) # Output: [('b', 1), ('a', 10), ('c', 22)] tidak berurutan
8
9 # Mengurutkan list dari tuple berdasarkan key (default sort pada elemen pertama dari tuple)
10 t.sort()
11 print("\nList tuple setelah diurutkan berdasarkan key:")
12 print(t) # Output: [('a', 10), ('b', 1), ('c', 22)]
13
14 # Penjelasan tambahan:
15 # Fungsi sort() akan mengurutkan berdasarkan elemen pertama dari tuple (yaitu key)
16 # Karena key adalah string, maka urutannya secara alfabet (ascending)
17
18 # Jika ingin mengurutkan berdasarkan value, bisa gunakan parameter key:
19 t_by_value = sorted(d.items(), key=lambda item: item[1])
20 print("\nList tuple diurutkan berdasarkan value (nilai):")
21 print(t_by_value) #Output: [('b', 1), ('a', 10), ('c', 22)]
```

Kata yang sering muncul

Program pada bagian ini dirancang untuk menghitung frekuensi kemunculan setiap kata dalam teks drama Romeo and Juliet (Act 2, Scene 2) yang tersimpan dalam file romeo-full.txt. Proses diawali dengan membaca file, membersihkan tanda baca, mengubah seluruh huruf menjadi huruf kecil, kemudian memisahkan setiap baris menjadi daftar kata. Semua kata disimpan ke dalam dictionary dengan kata sebagai key dan jumlah kemunculannya sebagai value.

```
2 import string
3 fhand = open('romeo-full.txt')
4 counts = dict()
5 for line in fhand:
6     line = line.translate(str.maketrans('', '', string.punctuation))
7     line = line.lower()
8     words = line.split()
9     for word in words:
10         if word not in counts:
11             counts[word] = 1
12         else:
13             counts[word] += 1
14
15 # urutkan dictionary by value
16 lst = list()
17 for key, val in list(counts.items()):
18     lst.append((val, key))
19
20 lst.sort(reverse=True)
21
22 for key, val in lst[:10]:
23     print(key, val)
```

Setelah seluruh teks selesai diproses, dictionary dikonversi menjadi list of tuples dalam format (jumlah kemunculan, kata), yang kemudian diurutkan secara menurun berdasarkan frekuensi. Program kemudian menampilkan 10 kata dengan kemunculan terbanyak beserta jumlahnya masing-masing.

Tuple sebagai kunci *dictionaries*

Karena tuple bersifat hashable, tuple dapat digunakan sebagai kunci (key) dalam dictionary—sesuatu yang tidak bisa dilakukan oleh list karena list bersifat mutable. Kemampuan ini sangat berguna ketika kita perlu menggunakan kombinasi beberapa nilai sebagai satu kunci tunggal, misalnya menggabungkan nama depan dan nama belakang seseorang.

Sebagai ilustrasi, kita dapat membangun sebuah direktori telepon yang menggunakan pasangan (nama belakang, nama depan) sebagai kunci dan nomor telepon sebagai nilainya. Saat melakukan iterasi pada dictionary tersebut, setiap kunci berupa tuple dapat langsung dipecah ke dalam dua variabel terpisah untuk kemudian ditampilkan sesuai kebutuhan.

```
1  # Mendefinisikan nilai-nilai untuk nama belakang, nama depan, dan nomor telepon
2  last = 'nendya'      # Variabel last menyimpan string 'nendya' sebagai nama belakang
3  first = 'dida'       # Variabel first menyimpan string 'dida' sebagai nama depan
4  number = '088112266' # Variabel number menyimpan string nomor telepon
5
6  # Membuat dictionary kosong untuk direktori telepon
7  directory = dict() # Inisialisasi dictionary kosong bernama directory
8
9  # Menggunakan tuple (last, first) sebagai key dalam dictionary
10 directory[last, first] = number
11 # Menyimpan data ke dictionary dengan key berupa tuple (last, first)
12 # Artinya key-nya adalah ('nendya', 'dida'), dan valuenya adalah nomor telepon
13
14 # Melakukan iterasi pada dictionary, key-nya adalah tuple (last, first)
15 for last, first in directory:
16     # Looping akan mengambil setiap key dari dictionary
17     # Setiap key adalah tuple (last, first), yang dipecah menjadi variabel last dan first
18
19     # Menampilkan nama depan, nama belakang, dan nomor telepon
20     print(first, last, directory[last, first])
21     # Mengakses value dictionary dengan key tuple (last, first)
22     # Mencetak: nama depan, nama belakang, dan nomor telepon sesuai pasangan key-value
```

BAGIAN 2: LATIHAN MANDIRI (60%)

Pada bagian ini anda menuliskan jawaban dari soal-soal Latihan Mandiri yang ada di modul praktikum. Jawaban anda harus disertai dengan source code, penjelasan dan screenshot output.

SOAL 1

https://github.com/grec12/71251235_nataliagrecia.git

```
1  def pengecekan(tA):
2      ada = tA[0]
3
4      for elemen in tA:
5          if elemen != ada:
6              return False
7      return True
8
9  tA = (90, 90, 90, 90)
10 print(pengecekan(tA))
```

True

Penjelasan kode:

- Definisikan sebuah fungsi yang menerima parameter **tA**.
- Buatlah sebuah variabel untuk menampung elemen paling awal (indeks kiri) dari **tA** yang akan digunakan sebagai pembanding dalam proses perulangan.
- Lakukan iterasi pada setiap elemen di **tA** untuk mengecek kesesuaian antara elemen paling kiri dengan elemen paling kanan.
- Apabila hasil pengecekan tidak cocok, fungsi akan mengembalikan nilai *false*. Sebaliknya, jika keduanya sama, fungsi akan mengembalikan nilai *true*.
- Terakhir, cetak output dari fungsi tersebut.

SOAL 2

```
1 data = ('Matahari Bhakti Nendya', '22064091', 'Bantul, DIYogyakarta')
2
3 print("NIM : ", data[1])
4 print("NAMA : ", data[0])
5 print("ALAMAT : ", data[2])
6
7 nim = tuple(data[1])
8 print("\nNIM: ", nim)
9
10 nama_depan = tuple(data[0].split()[0])
11 print("NAMA DEPAN: ", nama_depan[1:])
12
13 nama_terbalik = tuple(reversed(data[0].split()))
14 print("NAMA TERBALIK: ", nama_terbalik)
```

```
NIM : 22064091
NAMA : Matahari Bhakti Nendya
ALAMAT : Bantul, DIYogyakarta

NIM: ('2', '2', '0', '6', '4', '0', '9', '1')
NAMA DEPAN: ('a', 't', 'a', 'h', 'a', 'r', 'i')
NAMA TERBALIK: ('Nendya', 'Bhakti', 'Matahari')
```

Penjelasan kode:

- Inisiasikan sebuah variabel yang berisi list data.
- Cetak elemen pada indeks ke-0 untuk memunculkan NIM.
- Gunakan cara yang sama (pemanggilan indeks list) untuk menampilkan nama serta alamat.
- Untuk menampilkan NIM sesuai format yang diminta, konversikan list tersebut menjadi tuple agar setiap karakternya terpisah secara otomatis berdasarkan indeks. Lakukan hal serupa untuk menampilkan karakter nama.
Perbedaannya, pada bagian nama, karakter pertama (indeks paling kiri) tidak ikut ditampilkan. Oleh karena itu, gunakan slicing indeks **[1:]** untuk memanggil karakter mulai dari indeks kedua hingga akhir.
- Untuk menampilkan nama dengan urutan terbalik, manfaatkan fungsi **reverse** yang akan memutar posisi karakter dari depan ke belakang.
- Cetak seluruh hasilnya.

SOAL 3

```
1  in_file = input("Enter a file name: ")
2
3  try:
4      buka = open(in_file)
5  except:
6      print("File tidak ditemukan")
7      quit()
8
9  itung_jam = dict()
10 for line in buka:
11     if line.startswith("From "):
12         parts = line.split()
13         waktu = parts[5]
14         jam = waktu.split(':')[0]
15         itung_jam[jam] = itung_jam.get(jam, 0) + 1
16
17 hasil = sorted(itung_jam.items())
18 for jam, jumlah in hasil:
19     print(jam, jumlah)
```

```
Enter a file name: mbox-short.txt
04 3
06 1
07 1
09 2
10 3
11 6
14 1
15 2
16 4
17 2
18 1
19 1
```

Penjelasan kode:

- Buat blok **try-except** di awal untuk memvalidasi keberadaan file. Jika file tidak ditemukan, cetak pemberitahuan bahwa file tidak ada; namun jika file ada, lanjutkan ke proses berikutnya.
- Siapkan sebuah dictionary kosong untuk menampung hasil dari proses perulangan.

- Lakukan iterasi pada setiap baris di dalam file, kemudian periksa apakah baris tersebut diawali dengan kata "from".
- Buat variabel baru untuk memecah baris tersebut menggunakan metode **split**.
- Akses bagian waktu pada hasil split, lalu pecah lagi menggunakan pemisah ":" dan ambil elemen pada indeks ke-0 untuk mendapatkan nilai jamnya.
- Panggil dictionary dengan menggunakan nilai jam tersebut sebagai kunci. Jika jam belum tercatat di dictionary, set nilai awalnya menjadi 0.
- Tambahkan nilai 1 ke dalam dictionary sebagai penanda bahwa jam tersebut baru saja muncul satu kali. Ulangi proses kondisi ini hingga seluruh baris pada file habis.
- Buat variabel baru untuk mengurutkan hasil dari perulangan tersebut.
- Terakhir, cetak isi dictionary dengan mengakses pasangan *key-value* (jam dan jumlah kemunculannya) dari variabel hasil yang sudah diurutkan.